

Lección 22: Docs as Code

Documentación que Vive con el Código

Agenda

- ¿Por qué Docs as Code?
 - El Problema de la Deriva de Documentación
 - Tipos de Documentación
 - Docs en Control de Versiones
 - Documentación Viva
 - Documentación Asistida por IA
-

El Problema Tradicional

Documentación separada del código:

Code:

- ✓ Actualizado constantemente
- ✓ Versionado en Git
- ✓ Code reviews
- ✓ CI/CD automated

Docs:

- ✗ Wiki separado
- ✗ Google Docs
- ✗ Confluence desactualizado
- ✗ Manual, sin automation

Resultado: Docs desactualizados en semanas

Deriva de Documentación

El ciclo de muerte de la documentación:

Semana 1: Docs escritos, 100% precisos
Semana 2: Código cambia, docs no se actualizan
Semana 4: Docs 20% desactualizados
Semana 8: Docs 50% incorrectos
Semana 12: Nadie confía en los docs
Semana 16: Developers preguntan en Slack
Semana 20: Docs completamente ignorados

Tiempo para Leer el Código: Cada vez

Costo:

- Onboarding lento
- Preguntas repetidas
- Decisiones sin contexto
- Silos de conocimiento

Docs as Code: Filosofía

Trata los docs como código:

- ✓ Mismo repositorio que el código
- ✓ Mismo proceso de revisión
- ✓ Mismo pipeline CI/CD
- ✓ Mismo versionado
- ✓ Mismas herramientas (VSCode, Git)

Beneficios:

- Docs actualizados con el código
 - Revisores verifican ambos
 - Validación automatizada
 - Nunca desincronizados
 - Única fuente de verdad
-

Tipos de Documentación

1. Documentación de API (para desarrolladores):

```
/**
 * Calcula el precio total incluyendo impuestos y descuentos
 *
 * @param items - Items del carrito con cantidades
 * @param taxRate - Tasa de impuesto como decimal (0.21 = 21%)
 * @param discountCode - Código de descuento opcional
 * @returns Precio total en centavos
 *
 * @example
 * const total = calculateTotal(
 *   [{ id: 1, price: 1000, quantity: 2 }],
 *   0.21,
 *   'SUMMER20'
 * )
 * // retorna 1936 (20€ - 20% descuento + 21% impuesto)
 */
export function calculateTotal(items: CartItem[], taxRate: number, discount
  // ...
}
```

Tipos de Documentación (cont.)

2. Docs de Arquitectura (ADRs - Architecture Decision Records):

ADR-001: Usar Zustand para Gestión de Estado

Estado

Aceptado

Contexto

El carrito de compras necesita estado global. Opciones: Redux, Zustand, Con

Decisión

Usar Zustand porque:

- Boilerplate mínimo
- TypeScript de primera clase
- Soporte DevTools
- 3KB de bundle size

Consecuencias

- Desarrollo rápido
 - Testing fácil
- * El equipo necesita aprender nueva librería

Tipos de Documentación (cont.)

3. Documentación de Componentes (Storybook):

```
/**
 * Componente de botón principal para acciones principales
 */
export const Button = ({ variant, children, onClick }: ButtonProps) => {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  )
}

// Historia de Storybook
export default {
  title: 'Components/Button',
  component: Button,
}

export const Primary = {
  args: {
    variant: 'primary',
    children: 'Agregar al Carrito',
  },
}
```

4. Guías de Usuario (README, Getting Started)

JSDoc / TSDoc: Documentación Inline (1/2)

Sin documentación:

```
export function applyDiscount(price: number, code: string) {
  const discount = DISCOUNT_CODES[code]
  if (!discount) return price
  return price - (price * discount.percentage) / 100
}
```

```
// Preguntas del desarrollador:
// - ¿Qué unidades para price? ¿Centavos? ¿Dólares?
// - ¿Qué pasa si el código no existe?
// - ¿Cuál es el valor de retorno?
// - ¿Hay efectos secundarios?
```

JSDoc / TSDoc: Documentación Inline (2/2)

Con JSDoc:

```
/**
 * Aplica un código de descuento a un precio
 *
 * @param price - Precio en centavos (ej., 1000 = $10.00)
 * @param code - Código de descuento (ej., "SUMMER20")
 * @returns Precio con descuento en centavos, o el original si el código es
 *
 * @example
 * applyDiscount(1000, "SUMMER20") // 800 (20% de descuento)
 * applyDiscount(1000, "INVALID") // 1000 (sin cambios)
 */
export function applyDiscount(price: number, code: string): number {
  // ...
}
```

Documentación Viva

Código autodocumentado:

//  Necesita comentarios para entender

```
function calc(i, t, d) {  
  let s = 0  
  i.forEach(x => (s += x.p * x.q))  
  if (d) s = s * (1 - d.pct)  
  return s + s * t  
}
```

//  El código ES la documentación

```
function calculateCartTotal(items: CartItem[], taxRate: number, discount?:  
  const subtotal = items.reduce((sum, item) => sum + item.price * item.quan  
  
  const afterDiscount = discount ? subtotal * (1 - discount.percentage) : s  
  
  const total = afterDiscount + afterDiscount * taxRate  
  
  return total  
}
```

Documentación Viva: Tipos

Definiciones de tipos como documentación:

```
/**  
 * Item del carrito de compras  
 */  
export interface CartItem {  
  /** ID del producto */  
  id: number  
  
  /** Nombre del producto para mostrar */  
  name: string  
  
  /** Precio en centavos (1000 = $10.00) */  
  price: number  
  
  /** Cantidad seleccionada (min: 1, max: 99) */  
  quantity: number  
  
  /** URL de imagen opcional */  
  imageUrl?: string  
}
```

// TypeScript + JSDoc = Documentación con autocompletado!

README: Documentación del Proyecto (1/2)

Plantilla Mínima de README:

Carrito de Compras

Carrito de compras e-commerce con React + TypeScript

Inicio Rápido

```
pnpm install
pnpm dev
```

Arquitectura

- **Frontend**: React 18 + Vite + TypeScript
- **Estado**: Zustand
- **Testing**: Vitest + Testing Library + Playwright
- **Observabilidad**: Sentry

README: Documentación del Proyecto (2/2)

Estructura del Proyecto

```
src/
├── components/ # Componentes UI
├── hooks/ # Custom React hooks
├── store/ # Estado Zustand
├── utils/ # Funciones puras
└── api/ # Cliente API
```

Testing

```
pnpm test # Unit + integration
pnpm test:e2e # End-to-end
pnpm test:coverage # Reporte de cobertura
```

Contribuir

Registros de Decisiones de Arquitectura (ADRs)

Plantilla:

ADR-XXX: [Título de la Decisión]

Fecha: YYYY-MM-DD

Estado: [Propuesto | Aceptado | Deprecado | Reemplazado]

Contexto

¿Cuál es el problema? ¿Por qué necesitamos tomar una decisión?

Opciones Consideradas

1. Opción A
2. Opción B
3. Opción C

Decisión

Elegimos Opción B

Justificación

- Razón 1
- Razón 2
- Razón 3

Consecuencias

Positivas

- Beneficio 1
- Beneficio 2

Negativas

- Compromiso 1

- Compromiso 2

Referencias

- Link a RFC
- Link a Spike

Ejemplo ADR: Carrito de Compras (1/2)

ADR-003: Estrategia de Persistencia del Estado del Carrito

Fecha: 2024-01-15

Estado: Aceptado

Contexto

Los usuarios esperan que el carrito persista entre sesiones de navegador y
Opciones: localStorage, sessionStorage, base de datos, cookies.

Decisión

Usar **localStorage** para usuarios anónimos, **base de datos** para autent

Justificación

- 90% de usuarios navegan anónimamente
- localStorage: latencia cero, sin llamadas backend
- Base de datos: persistente entre dispositivos para usuarios logueados
- Enfoque híbrido: mejor rendimiento para la mayoría

Ejemplo ADR: Carrito de Compras (2/2) (1/2)

Consecuencias

Positivas

- Rápido para usuarios anónimos
- Persistente para usuarios logueados
- Sin carga del servidor para navegación

Negativas

- Carrito perdido si se limpia localStorage
- Necesita lógica de sincronización al login
- Estrategias de almacenamiento duplicadas

Ejemplo ADR: Carrito de Compras (2/2) (2/2)

Implementación

```
// Anónimo: localStorage
const saveCart = (cart) => {
  localStorage.setItem('cart', JSON.stringify(cart))
}
```

```
// Autenticado: API
const syncCart = async (cart) => {
  await fetch('/api/cart', {
    method: 'POST',
    body: JSON.stringify(cart)
  })
}
```

Docs en CI/CD

Validación automatizada:

```
# .github/workflows/docs.yml
name: Documentation

on: [pull_request]

jobs:
  docs-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Verificar cobertura JSDoc
        run: |
```

```
npm run docs:check
# Falla si funciones no tienen JSDoc
```

- name: Construir docs
run: npm run docs:build
Falla si docs tienen errores
- name: Verificar enlaces rotos
run: npm run docs:links
Falla si enlaces internos rotos
- name: Revisión ortográfica
run: npm run docs:spell
Advierte sobre errores ortográficos

Documentación Asistida por IA (1/2)

GitHub Copilot para JSDoc:

```
// Escribe la firma de la función
export function validateCartItem(item: CartItem): ValidationResult {

// Copilot sugiere:
/**
 * Valida un item del carrito asegurando que cumple las reglas de negocio
 *
 * Verifica:
 * - Cantidad está entre 1 y 99
 * - Precio es positivo
 * - ID de producto existe
 *
 * @param item - Item del carrito a validar
 * @returns Resultado de validación con errores si los hay
 */
```

Documentación Asistida por IA (2/2)

AI Agents para README:

Prompt: "Crea un README para un carrito de compras React con

TypeScript, Vitest, y Playwright. Incluye inicio rápido, arquitectura, y secciones de testing."

Output: README profesional en segundos

Mejores Prácticas de Documentación (1/2)

1. Mantenla cerca del código:

- ✓ src/utils/cart-operations.ts
- ✓ src/utils/cart-operations.test.ts
- ✓ src/utils/cart-operations.md (si es complejo)
- ✗ docs/wiki/cart-utils.md (lejos del código)

Mejores Prácticas de Documentación (2/2)

2. Documenta el “por qué”, no el “qué”:

```
// ✗ Qué (obvio del código)
// Iterar items y sumar precios
items.forEach(item => (total += item.price))

// ✓ Por qué (explica razonamiento)
// Precios almacenados en centavos para evitar errores de punto flotante
const totalInCents = items.reduce((sum, item) => sum + item.price, 0)
```

Mejores Prácticas de Documentación (3/3)

(1/2)

3. Ejemplos > Explicaciones:

```
/**
 * ✗ Explicación verbosa
 * Esta función toma un array de items del carrito y calcula
```

```

* el total iterando cada item, multiplicando el
* precio por cantidad, sumando todo, y retornando el resultado
*/

/**
*  Ejemplo claro
* Calcula el total del carrito
*
* @example
* calculateTotal([
*   { price: 1000, quantity: 2 },
*   { price: 500, quantity: 1 }
* ])
* // retorna 2500 ($25.00)
*/

```

Mejores Prácticas de Documentación (3/3) (2/2)

4. Actualiza docs en el mismo PR que el código:

Commit 1: Agregar feature de descuentos
 Commit 2: Actualizar JSDoc para función de descuentos
 Commit 3: Actualizar README con docs de descuentos

Todo en mismo PR → Nunca desincronizado

Carrito de Compras: Estructura de Docs

```

ai-course/
├── README.md                # Resumen del proyecto
├── CONTRIBUTING.md         # Cómo contribuir
├── docs/
│   ├── adr/                # Decisiones de arquitectura
│   │   ├── 001-zustand.md
│   │   ├── 002-vitest.md
│   │   └── 003-cart-persistence.md
│   ├── api/                # Documentación API
│   └── cart-api.md

```

```
|   └─ guides/                                # Guías de usuario
|       └─ getting-started.md
|       └─ testing-guide.md
└─ src/
    └─ components/
        └─ Button/
            └─ Button.tsx                    # Código del componente
            └─ Button.test.tsx              # Tests
            └─ Button.stories.tsx           # Docs Storybook
        └─ utils/
            └─ cart-operations.ts           # JSDoc inline
            └─ README.md                    # Resumen de utils
└─ .storybook/                             # Config Storybook
```

Ejercicio 1: JSDoc Completo para API Documentation

Prompt:

Actúa como un technical writer implementando Docs as Code con JSDoc/TSDoc.

CONTEXT0: JSDoc = standard de documentación inline en JavaScript/TypeScript código). `/** */` comment block antes de función/clase. Tags: `@param` (parámetro), `@returns` (valor retornado), `@throws` (errores que lanza), `@example` (ejemplo). IDE integration: VS Code muestra JSDoc en hover tooltip + autocomplete (no tools). TypeScript + JSDoc: `type` checking en JSDoc (valida que `@param` tipos con firma). Documentation drift problem: docs externos se desactualizan, JS actualiza con código (mismo PR). Living documentation: código autodocumenta agrega contexto que código no puede expresar (WHY, no WHAT). `@example` tag: que explicaciones largas (show, don't tell).

TAREA: Documenta `calculateTotal` con JSDoc completo siguiendo best practices

UBICACIÓN:

- Archivo: `src/shared/utils/calculateSubtotal.ts` (o `cart-operations.ts`)
- Función: `calculateTotal(items: CartItem[]): number`
- Agregar JSDoc block ANTES de `export function`

JSDOC STRUCTURE REQUERIDA:

1. Descripción breve (1 línea: qué hace la función)
2. `@param` tag: cada parámetro con tipo + descripción
3. `@returns` tag: qué retorna + unidades si aplica
4. `@throws` tag: errores que puede lanzar (si aplica)

5. @example tag: ejemplo concreto de uso con resultado

EJEMPLO PATTERN:

```
/**
 * [Breve descripción en 1 línea]
 *
 * @param items - Array de items en el carrito
 * @returns Total del carrito en [unidad]
 * @throws {Error} Si [condición de error]
 *
 * @example
 * const items = [{ price: 10, quantity: 2 }]
 * calculateTotal(items) // 20
 */
```

TESTING JSDOC:

1. Agregar JSDoc block completo
2. Abrir otro archivo TypeScript
3. Import calculateTotal
4. Hover sobre calculateTotal en código
5. Verificar que tooltip muestra:
 - Descripción completa
 - Todos los @param
 - @returns
 - @example renderizado

VALIDACIÓN: Hover en VS Code debe mostrar documentación completa con ejemplo

Aprende: JSDoc vive con código, se actualiza en mismo commit,

y se muestra en IDE automáticamente

Ejercicio 2: Actualizar README con Sección Testing

Prompt:

Actúa como un developer advocate actualizando README con Docs as Code princ

CONTEXTO: README.md = punto de entrada para developers (primera cosa que le as Code: README vive en mismo repo, se actualiza en mismo PR que código. RE structure: Quick Start → Installation → Architecture → Testing → Contributi Testing section crítica: muestra cómo verificar que proyecto funciona (onbo esencial). Package manager consistency: proyecto usa pnpm (NO npm), docs de reflejar comandos reales. Example output: muestra qué esperar al ejecutar c (reduce confusion, valida setup correcto). Markdown code blocks: bash para comandos shell, plain para output. Update together rule: si agregas tests, README en MISMO commit (previene documentation drift).

TAREA: Agrega sección "Testing" completa al README.md del proyecto.

UBICACIÓN:

- Archivo: README.md (root del proyecto)
- Posición: Después de *## Installation* o *## Quick Start*
- Antes de: *## Contributing* (si existe)

SECCIÓN STRUCTURE:

1. Heading: *## Testing*
2. Subsection: *### Ejecutar Tests con comandos pnpm*
3. Code block: Comandos bash con comentarios inline
4. Subsection: *### Output Esperado*
5. Example output: Muestra resultado exitoso con  y stats

COMANDOS A DOCUMENTAR:

- pnpm `test` (unit + integration tests)
- pnpm `test:coverage` (con coverage report)
- pnpm `test:e2e` (Playwright E2E tests)

MARKDOWN TEMPLATE:

Testing

Ejecutar Tests

Unit + Integration tests

pnpm `test`

Con coverage report

pnpm `test:coverage`

End-to-end tests

pnpm test:e2e

Output Esperado

- ✓ calculateTotal > should sum prices (2ms)
- ✓ ShoppingCart > should add items (15ms)
- ✓ E2E > complete checkout flow (1.2s)

Tests: [number] passed, [number] total
Coverage: [percentage]%

VALIDACIÓN:

1. Abrir README.md en GitHub preview o VS Code preview
2. Verificar que código blocks renderizan correctamente
3. Copy/paste comandos bash → deben ejecutar sin errores
4. Verificar que output example coincide con actual test output

Aprende: README actualizado en mismo PR que código

previene documentation drift

Puntos Clave

1. **Docs as Code:** Mismo repo, mismo proceso, mismo CI/CD
2. **Deriva de Documentación:** Docs separados se desactualizan rápido
3. **Tipos de Docs:** API (JSDoc), Arquitectura (ADRs), Componentes (Storybook)
4. **Documentación Viva:** Código autodocumentado + docs inline
5. **Validación CI/CD:** Verificaciones automatizadas para docs
6. **Asistida por IA:** AI agents aceleran la escritura
7. **Mantener Cerca:** Docs junto al código que documentan
8. **Ejemplos > Explicaciones:** Mostrar, no explicar
9. **Actualizar Juntos:** Código + docs en mismo PR